

TRƯỜNG ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN II

Nghiên cứu và xây dựng bộ kết xuất đồ họa
Path Tracing thời gian thực tăng tốc bằng CUDA

NGUYỄN XUÂN KHÔI

khoi.nx225022@sis.hust.edu.vn

Giảng viên hướng dẫn: TS. Ngô Thành Trung _____

Chữ kí GVHD

Khoa: Khoa học máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

LỜI CẢM ƠN

Đồ án này là kết quả của quá trình học tập, nghiên cứu và thực hành của bản thân tôi trong suốt thời gian qua. Để hoàn thành được đồ án, bên cạnh nỗ lực của cá nhân, tôi đã nhận được rất nhiều sự giúp đỡ và động viên từ thầy cô, gia đình và bạn bè.

Trước hết, tôi xin bày tỏ lòng biết ơn sâu sắc tới thầy giáo hướng dẫn, TS. Ngô Thành Trung. Thầy không chỉ truyền đạt những kiến thức nền tảng quý báu về đồ họa máy tính và tính toán song song, mà còn định hướng tư duy, kiên nhẫn góp ý và tạo điều kiện để tôi tự do theo đuổi ý tưởng của mình. Nếu không có sự hướng dẫn tận tình của thầy, đồ án này khó có thể hoàn thành.

Tôi cũng xin gửi lời cảm ơn chân thành tới các thầy cô trong Trường Công nghệ Thông tin và Truyền thông, Đại học Bách Khoa Hà Nội, vì những kiến thức và kinh nghiệm đã trang bị cho tôi trong suốt quá trình học tập.

Cuối cùng, tôi xin cảm ơn gia đình và bạn bè – những người đã luôn ở bên cạnh, ủng hộ và là điểm tựa tinh thần vững chắc cho tôi trong những lúc khó khăn nhất.

Tôi xin chân thành cảm ơn!

LỜI CAM KẾT

Họ và tên sinh viên: Nguyễn Xuân Khôi

Điện thoại liên lạc:

Email: khoi.nx225022@sis.hust.edu.vn

Lớp:

Hệ đào tạo:

Tôi – Nguyễn Xuân Khôi – cam kết Đồ án II là công trình nghiên cứu của bản thân tôi dưới sự hướng dẫn của TS. Ngô Thành Trung. Các kết quả nêu trong đồ án là trung thực, là thành quả của riêng tôi, không sao chép theo bất kỳ công trình nào khác. Tất cả những tham khảo trong đồ án – bao gồm hình ảnh, bảng biểu, số liệu và các câu từ trích dẫn – đều được ghi rõ ràng và đầy đủ nguồn gốc trong danh mục tài liệu tham khảo. Tôi xin hoàn toàn chịu trách nhiệm với dù chỉ một sao chép vi phạm quy chế của nhà trường.

Hà Nội, ngày tháng năm 2026

Tác giả đồ án

Nguyễn Xuân Khôi

TÓM TẮT NỘI DUNG ĐỒ ÁN

Hình ảnh được sinh từ máy tính đóng vai trò quan trọng trong nhiều ứng dụng của thế giới thực, từ lĩnh vực giải trí như điện ảnh, trò chơi điện tử đến các lĩnh vực kỹ thuật như kiến trúc và y học. Kết xuất dựa trên vật lý (physically based rendering) là lớp giải thuật mã hóa các định luật truyền ánh sáng thành mô hình toán học để tạo ra hình ảnh có độ chân thực cao. Trong số đó, Path Tracing là phương pháp tổng quát và chính xác, song có chi phí tính toán rất lớn do phải mô phỏng đường đi của hàng triệu tia sáng bằng tích phân Monte Carlo, dẫn tới thời gian kết xuất kéo dài và khó đạt tốc độ tương tác.

Trong đồ án này, tôi nghiên cứu và xây dựng một bộ kết xuất Path Tracing chạy song song trên kiến trúc CUDA của NVIDIA nhằm khai thác sức mạnh của GPU để đạt tốc độ kết xuất thời gian thực. Hệ thống hiện thực đầy đủ chu trình dò tia trên GPU, bao gồm: thuật toán Path Tracing với tích lũy theo thời gian và khử răng cưa; hệ thống vật liệu khuếch tán, kim loại và điện môi; mô hình camera thấu kính mỏng với hiệu ứng độ sâu trường ảnh và nhòe chuyển động; cơ chế cầu nối CUDA–OpenGL cho phép hiển thị trực tiếp từ bộ nhớ video; cùng bộ nạp cảnh theo định dạng Mitsuba 3 và giao diện điều khiển tương tác.

Kết quả cho thấy hệ thống có khả năng kết xuất các cảnh thử nghiệm ở tốc độ tương tác, đồng thời hội tụ dần về hình ảnh ít nhiễu khi camera giữ nguyên. Đồ án cũng chỉ ra các hướng tối ưu và mở rộng trong tương lai như cấu trúc tăng tốc BVH và các kỹ thuật giảm nhiễu nâng cao.

Sinh viên thực hiện

(Ký và ghi rõ họ tên)

ABSTRACT

Computer-generated images play an important role in many real-world applications, ranging from recreational fields such as cinema and video games to engineering fields such as architecture and medicine. Physically based rendering is the class of algorithms that codifies the physical laws of light transport into mathematical models in order to produce highly realistic images. Among these, Path Tracing is a general and accurate method, but it is computationally expensive because it must simulate the paths of millions of light rays through Monte Carlo integration, resulting in long rendering times and making interactive frame rates difficult to achieve.

In this project, I study and build a Path Tracer that runs in parallel on NVIDIA's CUDA architecture to exploit the computational power of the GPU and achieve real-time rendering. The system implements the complete ray tracing pipeline on the GPU, including: the Path Tracing algorithm with temporal accumulation and anti-aliasing; a material system supporting diffuse, metallic, and dielectric surfaces; a thin-lens camera model with depth-of-field and motion-blur effects; a CUDA–OpenGL interoperability mechanism that enables display directly from video memory; together with a Mitsuba 3 scene loader and an interactive control interface.

Experimental results show that the system is able to render the test scenes at interactive rates while progressively converging to a low-noise image when the camera remains static. The project also identifies future optimization and extension directions, such as bounding volume hierarchy (BVH) acceleration structures and advanced denoising techniques.

Student

(Signature and full name)

Mục lục

1	Giới thiệu đề tài	1
1.1	Đặt vấn đề	1
1.2	Mục tiêu của đề tài	1
1.3	Phạm vi và đối tượng nghiên cứu	1
1.4	Cấu trúc báo cáo	1
2	Nền tảng lý thuyết	2
2.1	Đại lượng trắc quang và phương trình kết xuất	2
2.1.1	Góc khối và bức xạ	2
2.1.2	Hàm phản xạ lưỡng hướng (BRDF)	3
2.1.3	Phương trình kết xuất	3
2.2	Ước lượng Monte Carlo	3
2.2.1	Bộ ước lượng cơ bản	3
2.2.2	Lấy mẫu theo tầm quan trọng	4
2.2.3	Công thức tích phân đường và hệ số suy giảm	4
2.2.4	Kết thúc đường đi và Russian Roulette	5
2.3	Lấy mẫu điểm ảnh và tích lũy theo thời gian	5
2.3.1	Lý thuyết lấy mẫu và hiện tượng răng cưa	5
2.3.2	Khử răng cưa bằng lấy mẫu ngẫu nhiên	6
2.3.3	Tích lũy theo thời gian	7
2.3.4	Hiệu chỉnh gamma	7
2.4	Mô hình vật liệu	7
2.4.1	Vật liệu khuếch tán (Lambertian)	7
2.4.2	Vật liệu kim loại (Metal)	8
2.4.3	Vật liệu điện môi (Dielectric)	8
2.5	Mô hình camera thấu kính mỏng	8
2.5.1	Phương trình đo của camera	8
2.5.2	Từ lỗ kim tới thấu kính mỏng	9
2.5.3	Sinh tia với khẩu độ hữu hạn	9
2.5.4	Hệ trục và nhòe chuyển động	10
2.6	Tìm giao tia–vật thể và cấu trúc tăng tốc	10
2.6.1	Giao tia–khối cầu	10
2.6.2	Cấu trúc tăng tốc	10
2.7	Kiến trúc lập trình song song CUDA	11

3	Triển khai hệ thống	12
3.1	Tổng quan kiến trúc hệ thống	12
3.2	Công nghệ và môi trường phát triển	12
3.3	Cầu nối CUDA–OpenGL và vòng lặp kết xuất	13
3.4	Hiện thực nhân Path Tracing trên CUDA	13
3.5	Hệ thống vật liệu	14
3.6	Mô hình camera	15
3.7	Quản lý cảnh và xây dựng thể giới trên GPU	15
3.8	Nạp cảnh từ Mitsuba 3	15
3.9	Tích lũy theo thời gian và hiệu chỉnh gamma	16
3.10	Giao diện điều khiển tương tác	16
4	Đánh giá	17
4.1	Môi trường thử nghiệm	17
4.2	Chất lượng kết xuất tổng thể	17
4.3	Tái tạo các loại vật liệu	18
4.4	Độ sâu trường ảnh	19
4.5	Nhòe chuyển động	19
4.6	Nhận xét chung	20
5	Kết luận và hướng phát triển	21
5.1	Kết luận	21
5.2	Hạn chế	21
5.3	Hướng phát triển trong tương lai	21
	Tài liệu tham khảo	22

Danh sách hình vẽ

2.1	Hình học tại một điểm bề mặt: bức xạ tới theo hướng ω_i trong góc khối vi phân $d\omega_i$, bức xạ phản xạ theo hướng ω_o , với pháp tuyến $\mathbf{n}$ và góc tới θ_i	3
2.2	Một đường đi ngẫu nhiên trong Path Tracing: tia xuất phát từ camera, tán xạ qua các điểm va chạm $\mathbf{p}_1, \mathbf{p}_2, \dots$ và kết thúc tại nguồn sáng; hệ số suy giảm β được nhân dồn dọc theo đường.	5
2.3	Phổ tần số sau khi lấy mẫu gồm phổ gốc và các bản sao lặp lại. (a) Khi thỏa điều kiện Nyquist (2.8), các bản sao tách rời nên khôi phục được tín hiệu. (b) Khi lấy mẫu thiếu (hoặc tín hiệu không giới hạn dải tần), các bản sao chồng lấn, gây hiện tượng răng cưa.	6
2.4	Mô hình thấu kính mỏng. Điểm nằm trên mặt phẳng lấy nét (xanh) hội tụ thành một điểm sắc nét trên cảm biến; điểm ngoài mặt phẳng lấy nét (đỏ) trải ra thành vòng tròn nhòe, tạo hiệu ứng độ sâu trường ảnh.	9
4.1	Ảnh kết xuất trên cảnh mật độ cao sau 1846 khung hình tích lũy.	18
4.2	Ba loại vật liệu Lambertian, điện môi và kim loại trên cùng một cảnh.	19
4.3	Hiệu ứng độ sâu trường ảnh ở hai cấu hình khẩu độ và khoảng lấy nét.	19
4.4	Nhòe chuyển động trên các khối cầu được gán vận tốc.	20

Chương 1

Giới thiệu đề tài

1.1 Đặt vấn đề

Kết xuất đồ họa (rendering) là quá trình tạo ra hình ảnh hai chiều từ mô tả của một cảnh ba chiều. Trong số các kỹ thuật kết xuất, *Path Tracing* là phương pháp dựa trên mô phỏng vật lý của ánh sáng, cho phép tạo ra hình ảnh có độ chân thực cao với các hiệu ứng như phản xạ, khúc xạ, đổ bóng mềm và chiếu sáng toàn cục (global illumination). Tuy nhiên, chi phí tính toán của Path Tracing rất lớn do phải mô phỏng hàng triệu tia sáng cho mỗi khung hình.

1.2 Mục tiêu của đề tài

Đề tài hướng tới việc nghiên cứu và xây dựng một bộ kết xuất Path Tracing chạy song song trên kiến trúc CUDA của NVIDIA nhằm đạt tốc độ tương tác (thời gian thực). Các mục tiêu cụ thể bao gồm:

- Hiện thực thuật toán Path Tracing trên GPU bằng CUDA.
- Xây dựng hệ thống vật liệu Lambertian, Metal, Dielectric và camera vật lý có độ sâu trường ảnh, nhòe chuyển động.
- Hiển thị thời gian thực qua cầu nối CUDA–OpenGL và tích lũy mẫu theo thời gian để khử nhiễu.
- Nạp cảnh theo định dạng Mitsuba 3 và cung cấp giao diện điều khiển tương tác.

1.3 Phạm vi và đối tượng nghiên cứu

1.4 Cấu trúc báo cáo

Báo cáo được tổ chức thành năm chương. Chương 1 giới thiệu đề tài. Chương 2 trình bày nền tảng lý thuyết. Chương 3 mô tả quá trình triển khai hệ thống. Chương 4 trình bày kết quả và đánh giá. Chương 5 đưa ra kết luận và hướng phát triển.

Chương 2

Nền tảng lý thuyết

Chương này trình bày cơ sở lý thuyết của hệ thống. Trước hết ta xây dựng các đại lượng trắc quang (radiometry) và phương trình kết xuất mô tả sự cân bằng năng lượng ánh sáng. Tiếp theo là phương pháp Monte Carlo dùng để ước lượng tích phân trong phương trình đó, kèm các kỹ thuật giảm phương sai. Phần sau trình bày mô hình hàm phản xạ (BRDF) cho ba loại vật liệu được hiện thực, mô hình camera thấu kính mỏng, bài toán tìm giao tia–vật thể, và cuối cùng là mô hình tính toán song song trên CUDA. Nội dung được tham khảo chủ yếu từ loạt sách *Ray Tracing in One Weekend* [5, 6] và sách *Physically Based Rendering* [4].

2.1 Đại lượng trắc quang và phương trình kết xuất

2.1.1 Góc khối và bức xạ

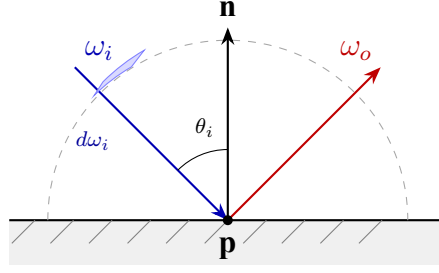
Để định lượng ánh sáng một cách độc lập với hình học, ta dùng khái niệm *góc khối* (solid angle). Một vùng trên mặt cầu đơn vị chắn bởi một hướng nhìn có độ lớn góc khối $d\omega$ (đơn vị steradian). Trong hệ tọa độ cầu, vi phân góc khối quanh hướng (θ, ϕ) là

$$d\omega = \sin \theta \, d\theta \, d\phi. \quad (2.1)$$

Đại lượng trung tâm của kết xuất dựa trên vật lý là *bức xạ* (radiance) L : công suất ánh sáng trên một đơn vị diện tích vuông góc, trên một đơn vị góc khối, ký hiệu $L(\mathbf{p}, \omega)$ tại điểm $\mathbf{p}$ theo hướng ω [4]. Bức xạ là đại lượng được *bảo toàn dọc theo tia* trong môi trường chân không và là giá trị mà mỗi tia trong path tracer mang theo. Từ bức xạ tới, *độ rọi* (irradiance) E trên một bề mặt có pháp tuyến $\mathbf{n}$ thu được bằng cách lấy tích phân theo bán cầu, có trọng số cosine:

$$E(\mathbf{p}) = \int_{\Omega} L_i(\mathbf{p}, \omega_i) (\omega_i \cdot \mathbf{n}) \, d\omega_i. \quad (2.2)$$

Thừa số cosine $(\omega_i \cdot \mathbf{n}) = \cos \theta_i$ phản ánh việc một chùm sáng tới xiên trải năng lượng trên diện tích lớn hơn nên đóng góp ít hơn.



Hình 2.1: Hình học tại một điểm bề mặt: bức xạ tới theo hướng ω_i trong góc khối vi phân $d\omega_i$, bức xạ phản xạ theo hướng ω_o , với pháp tuyến $\mathbf{n}$ và góc tới θ_i .

2.1.2 Hàm phản xạ lưỡng hướng (BRDF)

Cách một bề mặt phản xạ ánh sáng được mô tả bởi *hàm phân bố phản xạ lưỡng hướng* (Bidirectional Reflectance Distribution Function – BRDF), định nghĩa là tỉ số giữa bức xạ phản xạ theo hướng ω_o và độ rọi vi phân tới từ hướng ω_i :

$$f_r(\mathbf{p}, \omega_i, \omega_o) = \frac{dL_o(\mathbf{p}, \omega_o)}{L_i(\mathbf{p}, \omega_i) (\omega_i \cdot \mathbf{n}) d\omega_i}. \quad (2.3)$$

Một BRDF hợp lệ về mặt vật lý phải thỏa mãn hai tính chất: *tính đối xứng Helmholtz* $f_r(\omega_i, \omega_o) = f_r(\omega_o, \omega_i)$, và *bảo toàn năng lượng* – tổng năng lượng phản xạ không vượt quá năng lượng tới [4].

2.1.3 Phương trình kết xuất

Kết hợp BRDF với độ rọi tới, bức xạ rời khỏi điểm $\mathbf{p}$ theo hướng ω_o được cho bởi *phương trình kết xuất* do Kajiya đề xuất [2]:

$$L_o(\mathbf{p}, \omega_o) = L_e(\mathbf{p}, \omega_o) + \int_{\Omega} f_r(\mathbf{p}, \omega_i, \omega_o) L_i(\mathbf{p}, \omega_i) (\omega_i \cdot \mathbf{n}) d\omega_i, \quad (2.4)$$

trong đó L_e là bức xạ tự phát (vật liệu phát sáng), số hạng tích phân là bức xạ phản xạ. Đây là phương trình *Fredholm loại hai*: ẩn L xuất hiện ở cả hai vế vì bức xạ tới $L_i(\mathbf{p}, \omega_i)$ tại $\mathbf{p}$ chính là bức xạ đi ra $L_o(\mathbf{p}', -\omega_i)$ của điểm $\mathbf{p}'$ mà tia $(\mathbf{p}, \omega_i)$ chạm tới. Tính đệ quy này khiến lời giải giải tích là bất khả thi với cảnh tổng quát, do đó cần phương pháp số.

2.2 Ước lượng Monte Carlo

2.2.1 Bộ ước lượng cơ bản

Phương pháp Monte Carlo xấp xỉ một tích phân xác định bằng kỳ vọng thống kê. Với tích phân $I = \int g(x) dx$, nếu lấy N mẫu độc lập x_k theo mật độ xác suất $p(x)$ (với $p(x) > 0$ ở những nơi

$g(x) \neq 0$), bộ ước lượng Monte Carlo là

$$\langle I \rangle = \frac{1}{N} \sum_{k=1}^N \frac{g(x_k)}{p(x_k)}. \quad (2.5)$$

Bộ ước lượng này *không chệch* (unbiased): $\mathbb{E}[\langle I \rangle] = I$ với mọi N . Phương sai của nó giảm tỉ lệ $1/N$, nên *sai số chuẩn* (độ lệch chuẩn) giảm theo $O(1/\sqrt{N})$. Hệ quả thực tiễn: muốn giảm một nửa nhiễu cần gấp bốn lần số mẫu – đây là nguồn gốc của hiện tượng *nhiều hạt* (noise) đặc trưng và chi phí tính toán lớn của path tracing [6].

2.2.2 Lấy mẫu theo tầm quan trọng

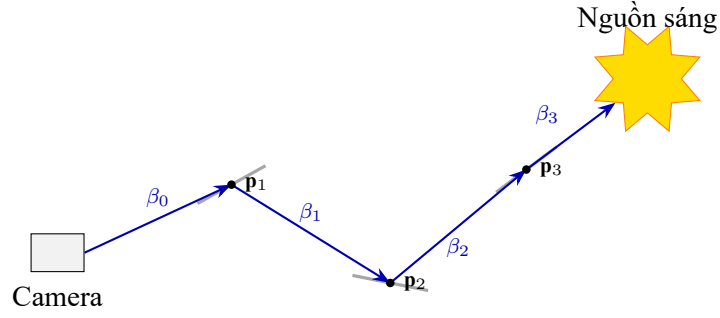
Phương sai phụ thuộc mạnh vào mức độ phù hợp giữa $p(x)$ và $g(x)$. Nếu chọn $p(x)$ tỉ lệ với $g(x)$ thì tỉ số g/p gần như hằng số và phương sai giảm mạnh; đây là ý tưởng *lấy mẫu theo tầm quan trọng* (importance sampling) [7]. Trong kết xuất, do tích phân (2.4) chứa thừa số cosine và BRDF, ta thường lấy mẫu hướng ω_i theo phân bố cosine hoặc theo hình dạng của BRDF thay vì đều trên bán cầu. Hệ thống hiện tại lấy mẫu khuếch tán theo phân bố cosine (Mục 2.4); việc lấy mẫu trực tiếp nguồn sáng (Next Event Estimation) là hướng mở rộng nêu ở Chương 5.

2.2.3 Công thức tích phân đường và hệ số suy giảm

Khai triển đệ quy phương trình (2.4) cho *công thức tích phân đường* (path integral): bức xạ tới camera là tổng đóng góp của mọi đường ánh sáng nối nguồn sáng tới mắt qua một số lần phản xạ. Path tracing ước lượng tổng này bằng cách, tại mỗi điểm va chạm, lấy *một* hướng tán xạ ngẫu nhiên và lần theo tia đó – tạo thành một đường đi ngẫu nhiên. Đóng góp của đường được tích lũy qua *hệ số suy giảm* (throughput) β , cập nhật nhân dồn sau mỗi va chạm:

$$\beta_n = \beta_{n-1} \frac{f_r(\mathbf{p}_n, \omega_i, \omega_o) (\omega_i \cdot \mathbf{n})}{p(\omega_i)}, \quad \beta_0 = 1. \quad (2.6)$$

Khi tia thoát khỏi cảnh và gặp nguồn sáng nền, bức xạ thu được bằng tích của bức xạ nền với β . Với cách lấy mẫu khuếch tán theo cosine, thừa số cosine và mẫu số $p(\omega_i)$ triệt tiêu nhau, khiến việc cập nhật β rút gọn về phép nhân với albedo – đúng như hiện thực trong mã nguồn (Chương 3).



Hình 2.2: Một đường đi ngẫu nhiên trong Path Tracing: tia xuất phát từ camera, tán xạ qua các điểm và chạm $\mathbf{p}_1, \mathbf{p}_2, \dots$ và kết thúc tại nguồn sáng; hệ số suy giảm β được nhân dồn dọc theo đường.

2.2.4 Kết thúc đường đi và Russian Roulette

Vì một đường có thể phản xạ vô hạn, cần điều kiện dừng. Cách đơn giản là giới hạn độ sâu tối đa, song điều này tạo ra sai lệch (bias) do bỏ qua các đường dài. Một giải pháp không chệch là *Russian Roulette* [4]: tại mỗi bước, kết thúc đường với xác suất q và chia phần còn lại cho $1 - q$ để giữ kỳ vọng không đổi. Hệ thống áp dụng giới hạn độ sâu (tối đa 50 lần dội) kết hợp dừng sớm khi β suy giảm gần bằng 0 – một xấp xỉ thực dụng cho chế độ thời gian thực.

2.3 Lấy mẫu điểm ảnh và tích lũy theo thời gian

2.3.1 Lý thuyết lấy mẫu và hiện tượng răng cưa

Quá trình tạo ảnh số thực chất là *lấy mẫu* (sampling) một hàm ảnh liên tục $f(x, y)$ – giá trị bức xạ tại mọi điểm trên mặt phẳng ảnh – tại một lưới điểm ảnh rời rạc, rồi *tái dựng* (reconstruction) ảnh hiển thị từ các mẫu đó. Cơ sở lý thuyết của quá trình này là phân tích Fourier [4].

Mỗi hàm f có một *phổ tần số* $F = \mathcal{F}\{f\}$ thu được qua biến đổi Fourier. Việc lấy mẫu đều với bước T tương đương nhân f với một chuỗi xung Dirac (Dirac comb) khoảng cách T . Theo định lý tích chập, trong miền tần số phép nhân này trở thành tích chập của phổ F với một chuỗi xung khoảng cách $1/T$, tạo ra *vô số bản sao* của phổ gốc lặp lại theo chu kỳ $1/T$:

$$\mathcal{F}\left\{f \cdot \sum_n \delta(x - nT)\right\} = F * \frac{1}{T} \sum_k \delta\left(\nu - \frac{k}{T}\right). \quad (2.7)$$

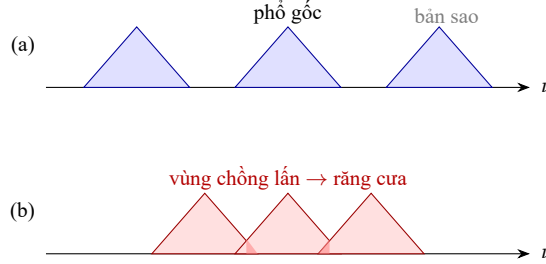
Để khôi phục f , ta lọc lấy bản sao trung tâm bằng một bộ lọc thông thấp lý tưởng. Điều này khả thi *nếu và chỉ nếu* các bản sao không chồng lấn, tức hàm phải *giới hạn dải tần* (band-limited) với tần số lớn nhất $\nu_{\max}$, và tần số lấy mẫu thỏa

$$\frac{1}{T} > 2\nu_{\max}. \quad (2.8)$$

Bất đẳng thức (2.8) chính là *định lý lấy mẫu Nyquist–Shannon*; ngưỡng $2\nu_{\max}$ gọi là *tần số*

Nyquist.

Trong kết xuất, điều kiện này gần như luôn bị vi phạm: biên hình học, bóng đổ sắc nét hay hoa văn texture tạo ra các *bước nhảy* có phổ trải tới vô hạn nên ảnh *không* giới hạn dải tần. Khi đó các bản sao phổ chồng lấn, năng lượng tần số cao “giả dạng” thành tần số thấp sau khi tái dựng, sinh ra các sai lệch nhìn thấy được gọi là *răng cưa* (aliasing): biên xiên bị bậc thang, hoa văn nhỏ biến thành vân moiré.



Hình 2.3: Phổ tần số sau khi lấy mẫu gồm phổ gốc và các bản sao lặp lại. (a) Khi thỏa điều kiện Nyquist (2.8), các bản sao tách rời nên khôi phục được tín hiệu. (b) Khi lấy mẫu thiếu (hoặc tín hiệu không giới hạn dải tần), các bản sao chồng lấn, gây hiện tượng răng cưa.

2.3.2 Khử răng cưa bằng lấy mẫu ngẫu nhiên

Có hai hướng chống răng cưa. Hướng thứ nhất là *tiền lọc* (prefiltering) – làm mượt hàm ảnh để giới hạn dải tần trước khi lấy mẫu – nhưng nhìn chung không khả thi vì hàm ảnh không có dạng giải tích. Hướng thứ hai, thực dụng hơn, là *tăng tần số lấy mẫu*: bắn nhiều tia trên mỗi điểm ảnh (*siêu lấy mẫu*, supersampling) để đẩy tần số Nyquist lên cao, thu hẹp vùng bị răng cưa.

Điều quan trọng là path tracer dùng *lấy mẫu ngẫu nhiên* (stochastic sampling) thay vì lưới đều. Với mẫu đều, năng lượng tần số cao chồng lấn tạo ra các vân răng cưa có cấu trúc; còn khi điểm lấy mẫu được làm *nhieve* ngẫu nhiên, năng lượng đó bị phân tán thành *nhieve* (noise) không cấu trúc. Mắt người ít nhạy với nhieve hơn nhiều so với các vân răng cưa có quy luật, nên đây là sự đánh đổi có lợi [4].

Về mặt hình thức, giá trị một điểm ảnh là tích phân của hàm ảnh nhân với một *hàm lọc tái dựng* r trên miền điểm ảnh, $I_{ij} = \int r(x, y) f(x, y) dx dy$. Path tracer ước lượng tích phân này bằng Monte Carlo (2.5) với các mẫu được làm *nhieve* ngẫu nhiên trong ô điểm ảnh. Mỗi khung dịch điểm bắn tia một lượng $(\xi_u, \xi_v) \in [0, 1)^2$,

$$s = \frac{i + \xi_u}{W}, \quad t = 1 - \frac{j + \xi_v}{H}, \quad (2.9)$$

với (i, j) là chỉ số điểm ảnh và $W \times H$ là độ phân giải (hệ thống dùng hàm lọc hộp, tức trọng số đều trong ô). Trung bình của nhiều mẫu vừa khử răng cưa biên, vừa hội tụ về giá trị tích phân đúng [5].

2.3.3 Tích lũy theo thời gian

Trong chế độ tương tác, mỗi khung chỉ kịp lấy một mẫu/điểm ảnh nên ảnh rất nhiễu. *Tích lũy theo thời gian* (temporal accumulation) tận dụng việc camera đứng yên: màu của các khung liên tiếp được cộng dồn rồi lấy trung bình,

$$C_{\text{out}} = \frac{1}{F} \sum_{f=1}^F C_f, \quad (2.10)$$

trong đó F là số khung kể từ lần đặt lại gần nhất. Đây chính là bộ ước lượng (2.5) được tích lũy dần theo thời gian: càng nhiều khung, F càng lớn, ảnh càng hội tụ (giảm nhiễu). Khi người dùng di chuyển camera, F được đặt lại về 1 để loại bỏ các mẫu cũ không còn hợp lệ.

2.3.4 Hiệu chỉnh gamma

Tính toán ánh sáng diễn ra trong không gian màu *tuyến tính*, nhưng màn hình kỳ vọng giá trị đã mã hóa gamma. Trước khi hiển thị, màu được hiệu chỉnh bằng $C' = C^{1/\gamma}$; hệ thống dùng $\gamma = 2$, tức lấy căn bậc hai từng kênh, giúp vùng tối hiển thị đúng độ sáng cảm nhận [5].

2.4 Mô hình vật liệu

Mỗi vật liệu định nghĩa một hàm *tán xạ*: cho tia tới, trả về tia tán xạ và hệ số suy giảm (attenuation). Hệ thống hiện thực ba mô hình kinh điển của loạt *Ray Tracing in One Weekend* [5].

2.4.1 Vật liệu khuếch tán (Lambertian)

Bề mặt khuếch tán lý tưởng có BRDF hằng số:

$$f_r = \frac{\rho}{\pi}, \quad (2.11)$$

với ρ là albedo (hệ số phản xạ) và hệ số π bảo đảm bảo toàn năng lượng khi lấy tích phân theo bán cầu. Để lấy mẫu hiệu quả, ta sinh hướng tán xạ theo *phân bố cosine* bằng một mẹo hình học: cộng pháp tuyến với một điểm ngẫu nhiên trên mặt cầu đơn vị,

$$\omega_i = \mathbf{n} + \mathbf{r}_{\text{unit}}, \quad \|\mathbf{r}_{\text{unit}}\| = 1. \quad (2.12)$$

Khi đó mật độ lấy mẫu $p(\omega_i) = \cos \theta_i / \pi$ khớp với thừa số cosine và BRDF, làm phép cập nhật throughput (2.6) rút gọn còn phép nhân với albedo ρ [6].

2.4.2 Vật liệu kim loại (Metal)

Kim loại phản xạ gương: tia phản xạ quanh pháp tuyến theo

$$\mathbf{r} = \mathbf{d} - 2(\mathbf{d} \cdot \mathbf{n}) \mathbf{n}, \quad (2.13)$$

với $\mathbf{d}$ là hướng tia tới đã chuẩn hóa. Để mô phỏng bề mặt nhám, hướng phản xạ được nhiễu loạn bằng một vector ngẫu nhiên trong quả cầu bán kính $fuzz$: $\mathbf{r}' = \mathbf{r} + fuzz \cdot \mathbf{r}_{rand}$. Giá trị $fuzz$ càng lớn, phản chiếu càng mờ; tia tán xạ chui xuống dưới bề mặt sẽ bị loại bỏ.

2.4.3 Vật liệu điện môi (Dielectric)

Vật liệu trong suốt (thủy tinh, nước) vừa phản xạ vừa khúc xạ. Phần khúc xạ tuân theo *định luật Snell*:

$$\eta \sin \theta_i = \eta' \sin \theta_t, \quad (2.14)$$

với η, η' là chiết suất hai môi trường, θ_i, θ_t là góc tới và góc khúc xạ. Khi $\frac{\eta}{\eta'} \sin \theta_i > 1$, phương trình (2.14) vô nghiệm với $\sin \theta_t$: xảy ra *phản xạ toàn phần*, tia buộc phải phản xạ theo (2.13).

Tỉ lệ năng lượng phản xạ so với khúc xạ được cho bởi *phương trình Fresnel*, vốn phức tạp vì phụ thuộc phân cực. Trong đồ họa, ta dùng *xấp xỉ Schlick* chính xác cao và rẻ về tính toán [4]:

$$R(\theta) = R_0 + (1 - R_0) (1 - \cos \theta)^5, \quad R_0 = \left(\frac{\eta - \eta'}{\eta + \eta'} \right)^2, \quad (2.15)$$

trong đó R_0 là hệ số phản xạ ở góc tới vuông góc. Thay vì tính cả hai tia phản xạ và khúc xạ rồi pha trộn, path tracer dùng cách *lấy mẫu ngẫu nhiên*: sinh một số $\xi \in [0, 1)$ và chọn phản xạ nếu $\xi < R(\theta)$, ngược lại thì khúc xạ. Theo nghĩa thống kê qua nhiều mẫu, tỉ lệ này tái tạo đúng hệ số Fresnel mà mỗi tia chỉ phải lần theo một nhánh [5].

2.5 Mô hình camera thấu kính mỏng

2.5.1 Phương trình đo của camera

Một cách hình thức, giá trị mà một điểm ảnh ghi nhận không phải là bức xạ của một tia đơn lẻ, mà là một *phép đo* (measurement) trên toàn bộ ánh sáng đi vào hệ quang học. Theo PBRT [4], đáp ứng I_j của điểm ảnh thứ j được cho bởi *phương trình đo* (measurement equation):

$$I_j = \int_{A_{\text{film}}} \int_{\Omega} W_e^{(j)}(\mathbf{p}, \omega) L_i(\mathbf{p}, \omega) |\cos \theta| d\omega dA(\mathbf{p}), \quad (2.16)$$

trong đó tích phân lấy trên diện tích phim A_{film} và bán cầu hướng Ω ; L_i là bức xạ tới, còn $W_e^{(j)}$ là *hàm tầm quan trọng* (importance) – đặc trưng độ nhạy của cảm biến với tia $(\mathbf{p}, \omega)$, đóng vai

trò đối ngẫu với bức xạ phát ra của một nguồn sáng. Nhiệm vụ của mô hình camera là sinh các tia sao cho việc lấy mẫu Monte Carlo tích phân (2.16) là hợp lệ.

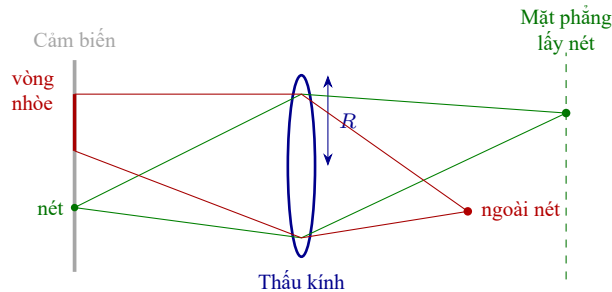
2.5.2 Từ lỗ kim tới thấu kính mỏng

Với camera *lỗ kim* (pinhole) lý tưởng, khẩu độ thu về một điểm: hàm tầm quan trọng W_e suy biến thành một xung Dirac theo hướng, nên mỗi điểm trên phim chỉ ứng với *một* tia duy nhất và mọi vật thể đều sắc nét bất kể độ sâu. Đây là lý do ảnh pinhole không có hiệu ứng nhòe của ống kính thật.

Để tái hiện *độ sâu trường ảnh* (depth of field), ta cần một khẩu độ hữu hạn. Mô hình *thấu kính mỏng* (thin lens) [4] xấp xỉ hệ quang học bằng một thấu kính lý tưởng độ dày bằng 0, tuân theo *phương trình thấu kính Gauss*:

$$\frac{1}{z'} - \frac{1}{z} = \frac{1}{f}, \quad (2.17)$$

với z là khoảng cách từ vật tới thấu kính, z' là khoảng cách ảnh, và f là tiêu cự. Phương trình (2.17) chỉ ra rằng các vật ở những khoảng cách z khác nhau hội tụ tại những mặt phẳng ảnh z' khác nhau. Do cảm biến đặt cố định, chỉ những vật nằm tại *mặt phẳng lấy nét* (khoảng cách *focus distance*) mới hội tụ thành điểm sắc nét; vật ở khoảng cách khác trải thành một đĩa gọi là *vòng tròn nhòe* (circle of confusion), gây hiệu ứng mờ nhòe. Bán kính vòng nhòe – và do đó độ mạnh của hiệu ứng – tỉ lệ với bán kính khẩu độ.



Hình 2.4: Mô hình thấu kính mỏng. Điểm nằm trên mặt phẳng lấy nét (xanh) hội tụ thành một điểm sắc nét trên cảm biến; điểm ngoài mặt phẳng lấy nét (đỏ) trải ra thành vòng tròn nhòe, tạo hiệu ứng độ sâu trường ảnh.

2.5.3 Sinh tia với khẩu độ hữu hạn

Tích phân (2.16) trên diện tích thấu kính được ước lượng bằng Monte Carlo: thay vì một tia, ta lấy mẫu một điểm trên đĩa thấu kính bán kính $R = \text{aperture}/2$. Quy trình sinh tia [4, 5] gồm ba bước:

1. Dựng tia chính đi từ tâm thấu kính qua điểm ảnh (s, t) .
2. Xác định *điểm lấy nét* là giao của tia chính với mặt phẳng lấy nét (cách camera một khoảng focus distance) – điểm này luôn sắc nét.

3. Lấy ngẫu nhiên một điểm trên đĩa thấu kính làm gốc tia mới, hướng tia mới đi qua đúng điểm lấy nét ở bước 2.

Vì mọi tia lấy mẫu đều đi qua cùng điểm lấy nét, vật tại mặt phẳng lấy nét vẫn sắc nét; còn vật ngoài mặt phẳng đó nhận các tia từ những vị trí thấu kính khác nhau nên bị nhòe, đúng như mô tả của vòng tròn nhòe. Khi khẩu độ bằng 0, mô hình thoái biến về camera lỗ kim.

2.5.4 Hệ trục và nhòe chuyển động

Trường nhìn dọc θ xác định chiều cao khung nhìn qua $h = \tan(\theta/2)$. Hệ trục camera trục chuẩn $(\mathbf{u}, \mathbf{v}, \mathbf{w})$ được dựng từ vị trí đặt máy (*lookfrom*), điểm nhìn (*lookat*) và vector hướng lên (*vup*):

$$\mathbf{w} = \frac{\text{lookfrom} - \text{lookat}}{\|\cdot\|}, \quad \mathbf{u} = \frac{\text{vup} \times \mathbf{w}}{\|\cdot\|}, \quad \mathbf{v} = \mathbf{w} \times \mathbf{u}. \quad (2.18)$$

Để mô phỏng *nhòe chuyển động* (motion blur), phương trình đo (2.16) được mở rộng thêm một tích phân theo *thời gian phơi sáng*. Hệ thống ước lượng tích phân này bằng cách gán cho mỗi tia một mốc thời gian ngẫu nhiên trong khoảng $[t_0, t_1]$; vật thể chuyển động được nội suy vị trí tuyến tính theo thời gian của tia, nên ảnh tích lũy thể hiện vệt mờ tự nhiên [5].

2.6 Tìm giao tia–vật thể và cấu trúc tăng tốc

2.6.1 Giao tia–khối cầu

Phép toán cốt lõi của mỗi bước dò tia là tìm giao điểm gần nhất giữa tia và vật thể. Một tia tham số hóa $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$ giao khối cầu tâm $\mathbf{c}$, bán kính r khi $\|\mathbf{r}(t) - \mathbf{c}\|^2 = r^2$. Khai triển cho phương trình bậc hai $at^2 + bt + c = 0$ với

$$a = \mathbf{d} \cdot \mathbf{d}, \quad b = 2\mathbf{d} \cdot (\mathbf{o} - \mathbf{c}), \quad c = (\mathbf{o} - \mathbf{c}) \cdot (\mathbf{o} - \mathbf{c}) - r^2. \quad (2.19)$$

Biệt thức $\Delta = b^2 - 4ac$ âm nghĩa là không có giao; ngược lại, nghiệm nhỏ hơn nằm trong khoảng $[t_{\min}, t_{\max}]$ cho điểm va chạm gần nhất. Pháp tuyến tại điểm giao $\mathbf{p}$ là $\mathbf{n} = (\mathbf{p} - \mathbf{c})/r$. Cận dưới $t_{\min}$ dương nhỏ giúp tránh *shadow acne* – hiện tượng tia thứ cấp tự giao lại bề mặt vừa xuất phát do sai số dấu phẩy động [5].

2.6.2 Cấu trúc tăng tốc

Khi cảnh có n vật thể, duyệt tuyến tính toàn bộ danh sách tốn $O(n)$ cho mỗi tia, không khả thi với cảnh phức tạp. Các *cấu trúc tăng tốc* như cây phân cấp khối bao (Bounding Volume Hierarchy – BVH) bao mỗi nhóm vật thể trong một khối bao (thường là hộp căn trục AABB) và tổ chức thành cây nhị phân. Nhờ kiểm tra giao với khối bao trước, ta loại bỏ sớm những nhánh tia không

thể chậm, giảm độ phức tạp trung bình xuống $O(\log n)$. Chất lượng cây phụ thuộc cách phân hoạch; *Surface Area Heuristic* (SAH) là tiêu chí phổ biến để chọn mặt phân chia tối ưu [4]. Phiên bản hiện tại dùng duyệt tuyến tính; tích hợp BVH là hướng tối ưu được bàn ở Chương 5.

2.7 Kiến trúc lập trình song song CUDA

CUDA là nền tảng tính toán song song đa dụng của NVIDIA, cho phép tận dụng hàng nghìn nhân của GPU [3]. Path tracing có tính *song song dữ liệu* lý tưởng: mỗi điểm ảnh được tính độc lập, hoàn toàn phù hợp mô hình SIMT (Single Instruction, Multiple Threads) của GPU.

Trong mô hình CUDA, công việc được tổ chức thành lưới (grid) gồm nhiều khối (block), mỗi khối chứa nhiều luồng (thread) chạy đồng thời theo nhóm 32 luồng (warp). Hàm chạy trên GPU gọi là *kernel*; khi khởi chạy, mỗi luồng xác định điểm ảnh phụ trách qua chỉ số toàn cục tính từ `blockIdx`, `blockDim` và `threadIdx`. Vì mỗi luồng cần một dãy số ngẫu nhiên độc lập cho Monte Carlo, hệ thống dùng thư viện `cuRAND` với hạt giống riêng cho từng điểm ảnh. Kích thước khối (ví dụ $8 \times 8 = 64$ luồng) ảnh hưởng trực tiếp tới mức độ chiếm dụng (occupancy) và hiệu năng. Một điểm cần lưu ý là sự *phân kỳ luồng* (warp divergence): khi các tia trong cùng warp rẽ nhánh khác nhau (ví dụ chạm vật liệu khác loại), GPU phải thực thi tuần tự các nhánh, làm giảm hiệu suất – đây là đặc thù quan trọng khi tối ưu path tracer trên GPU.

Chương 3

Triển khai hệ thống

Chương này mô tả cách hiện thực các khái niệm ở Chương 2 thành một hệ thống chạy được: kiến trúc tổng thể, cầu nối giữa CUDA và OpenGL để hiển thị thời gian thực, nhân (kernel) Path Tracing, hệ thống vật liệu và camera, quản lý cảnh trên GPU, bộ nạp cảnh Mitsuba 3 và giao diện điều khiển tương tác.

3.1 Tổng quan kiến trúc hệ thống

Hệ thống được chia thành ba tầng phối hợp với nhau trong mỗi khung hình:

- **Tầng host (CPU, C++):** quản lý vòng lặp ứng dụng, cấp phát bộ nhớ GPU, nạp cảnh, xử lý nhập liệu và điều khiển camera.
- **Tầng device (GPU, CUDA):** thực thi nhân Path Tracing, tính màu cho từng điểm ảnh và ghi kết quả trực tiếp vào bộ nhớ video (VRAM).
- **Tầng hiển thị (OpenGL + ImGui):** đưa ảnh từ VRAM lên màn hình và vẽ bảng điều khiển tương tác.

Điểm mấu chốt về hiệu năng là dữ liệu ảnh *không* phải đi vòng qua CPU: CUDA ghi thẳng vào bộ đệm mà OpenGL dùng để hiển thị, nhờ cơ chế CUDA–OpenGL Interop trình bày ở Mục 3.3.

3.2 Công nghệ và môi trường phát triển

Hệ thống được xây dựng bằng C++ và CUDA, cấu hình biên dịch qua CMake.

- Ngôn ngữ: C++ (tầng host) và CUDA C (nhân thiết bị).
- CUDA Toolkit phiên bản ≥ 11.0 , biên dịch bằng `nvcc`.
- CMake phiên bản ≥ 3.18 .
- Hiển thị: OpenGL kết hợp GLFW (quản lý cửa sổ, nhập liệu).
- Giao diện người dùng: thư viện ImGui.
- Sinh số ngẫu nhiên trên GPU: thư viện `cuRAND`.
- Phân tích cảnh Mitsuba: thư viện `tinyxml2`.

3.3 Cầu nối CUDA–OpenGL và vòng lặp kết xuất

Để hiển thị thời gian thực mà không tốn chi phí sao chép qua lại giữa CPU và GPU, hệ thống dùng cơ chế *interop*. Một *Pixel Buffer Object* (PBO) được tạo bởi OpenGL nằm trên VRAM, sau đó đăng ký với CUDA bằng `cudaGraphicsGLRegisterBuffer`. Từ đó, CUDA có thể ghi thẳng kết quả vào PBO, còn OpenGL dán PBO lên một texture để hiển thị.

Trong mỗi vòng lặp khung hình, hệ thống thực hiện bốn bước:

1. Cập nhật giao diện ImGui và camera theo thao tác chuột/bàn phím; nếu có thay đổi thì đồng bộ camera xuống GPU và đặt lại bộ đếm tích lũy.
2. Khóa (map) PBO, lấy con trỏ VRAM rồi gọi nhân kết xuất để tô màu toàn bộ điểm ảnh; sau đó nhả khóa (unmap).
3. OpenGL sao chép PBO vào texture (bằng DMA phần cứng) và vẽ một hình chữ nhật phủ kín cửa sổ.
4. Vẽ đè giao diện ImGui và hoán đổi bộ đệm (double buffering).

3.4 Hiện thực nhân Path Tracing trên CUDA

Nhân chính `render_anim_kernel` ánh xạ mỗi luồng GPU tới một điểm ảnh qua chỉ số toàn cục. Mỗi luồng nạp trạng thái ngẫu nhiên riêng, sinh dịch chuyển khử răng cưa, dựng tia từ camera rồi gọi hàm `ray_color` để dò đường:

```
1 int i = threadIdx.x + blockIdx.x * blockDim.x;
2 int j = threadIdx.y + blockIdx.y * blockDim.y;
3 if (i >= width || j >= height) return;
4 int pixel_index = j * width + i;
5
6 curandState local_rand_state = d_rand_state[pixel_index];
7 float s = float(i + curand_uniform(&local_rand_state)) / width;
8 float t = 1.0f - float(j + curand_uniform(&local_rand_state)) / height;
9
10 Ray ray = (*d_camera)->get_ray(s, t, &local_rand_state);
11 Color pixel_color = ray_color(ray, d_world, &local_rand_state);
```

Listing 3.1: Ánh xạ luồng–điểm ảnh và dựng tia (rút gọn).

Hàm `ray_color` hiện thực dò đường theo kiểu lặp (thay cho đệ quy, vốn tốn ngăn xếp trên GPU). Hệ số suy giảm `cur_attenuation` (β) được nhân dồn sau mỗi va chạm; vòng lặp dừng khi tia thoát ra nền trời, khi vật liệu không tán xạ, hoặc khi đạt số lần dội tối đa (50):

```
1 Color cur_attenuation(1, 1, 1);
2 for (int i = 0; i < 50; i++) {
3     hit_record rec;
4     if ((*world)->hit(cur_ray, 0.001f, 10000.0f, rec)) {
```

```

5 Ray scattered; Vec3 attenuation;
6 if (scatter(cur_ray, rec, attenuation, scattered, rand)) {
7     cur_attenuation = cur_attenuation * attenuation; // beta_n = beta_{n
      -1} * f_r
8     cur_ray = scattered;
9 } else return Color(0, 0, 0);
10 } else { // tia thoát ra bầu trời
11     float a = 0.5f * (unit_vector(cur_ray.direction()).y() + 1.0f);
12     Color sky = (1 - a) * Color(1, 1, 1) + a * Color(0.5f, 0.7f, 1.0f);
13     return sky * cur_attenuation;
14 }
15 }
16 return Color(0, 0, 0);

```

Listing 3.2: Vòng dò đường lặp với throughput.

Giá trị cận dưới $t_{\min} = 0,001$ giúp tránh hiện tượng *shadow acne* (tia tự va lại bề mặt vừa xuất phát do sai số dấu phẩy động).

3.5 Hệ thống vật liệu

Vật liệu được biểu diễn bằng một struct `Material` gọn nhẹ gồm kiểu vật liệu, albedo, hệ số nhám fuzz và chiết suất `ir`. Cách dùng struct phẳng (thay vì lớp ảo) giúp truyền dữ liệu xuống GPU thuận lợi. Hàm `scatter` phân nhánh theo kiểu vật liệu để xác định tia tán xạ:

- **Lambertian:** hướng tán xạ là pháp tuyến cộng vector đơn vị ngẫu nhiên; nếu hướng suy biến thì loại bỏ.
- **Metal:** phản xạ gương quanh pháp tuyến, cộng nhiễu bán kính fuzz; chỉ giữ tia phản xạ ra ngoài bề mặt.
- **Dielectric:** xét điều kiện phản xạ toàn phần và xác suất Schlick (2.15) để chọn giữa phản xạ và khúc xạ (định luật Snell).

```

1 float ratio = front_face ? (1.0f / mat.ir) : mat.ir;
2 float cos_theta = fabs(dot(normal, unit_vector(r_in.direction())));
3 bool cannot_refract = ratio * sqrtf(1 - cos_theta*cos_theta) > 1.0f;
4 if (cannot_refract || schlick(cos_theta, mat.ir) > random_float(rand))
5     scatter_direction = reflect(r_in.direction(), out_normal);
6 else
7     scatter_direction = refract(unit_vector(r_in.direction()), ratio,
      out_normal);

```

Listing 3.3: Quyết định phản xạ/khúc xạ cho điện môi (rút gọn).

3.6 Mô hình camera

Lớp Camera tiền tính các vector khung nhìn từ vị trí đặt máy, điểm nhìn, trường nhìn, tỉ lệ khung hình, khẩu độ và khoảng lấy nét. Hàm `get_ray` sinh tia cho tọa độ ảnh (s, t) : điểm xuất phát được dịch ngẫu nhiên trên đĩa thấu kính (tạo độ sâu trường ảnh), và mốc thời gian của tia được lấy ngẫu nhiên trong $[t_0, t_1]$ phục vụ nhòe chuyển động.

```
1 Vec3 rd = lens_radius * random_in_unit_disk(rand);
2 Vec3 offset = u * rd.x() + v * rd.y();
3 float time = time0 + random_float(rand) * (time1 - time0);
4 return Ray(origin + offset,
5           lower_left_corner + s*horizontal + t*vertical - origin - offset,
6           time);
```

Listing 3.4: Sinh tia với độ sâu trường ảnh và mốc thời gian.

3.7 Quản lý cảnh và xây dựng thế giới trên GPU

Cảnh trên host được mô tả bằng danh sách `SphereData` (tâm, bán kính, vật liệu, thông tin chuyển động). Dữ liệu này được sao chép xuống GPU, sau đó nhân `create_world_kernel` khởi tạo các đối tượng `Sphere` hoặc `MovingSphere` bằng *new ngay trên thiết bị* và gom vào một `HittableList`. Cách này cho phép sử dụng đa hình (hàm ảo `hit`) trực tiếp trong nhân GPU.

Việc tìm giao hiện được `HittableList` thực hiện bằng duyệt tuyến tính, luôn giữ lại giao điểm gần nhất bằng cách thu hẹp dần $t_{\max}$. Mỗi `Sphere` giải phương trình bậc hai như ở Chương 2 và điền bản ghi va chạm (vị trí, pháp tuyến, vật liệu). Đây là điểm còn có thể tối ưu bằng BVH (xem Chương 5).

3.8 Nạp cảnh từ Mitsuba 3

Để tái sử dụng các cảnh chuẩn, hệ thống hiện thực `MitsubaLoader` phân tích tệp `scene.xml` theo định dạng Mitsuba 3 [1] bằng `tinyxml2`. Bộ nạp duyệt các thẻ con của `<scene>` và xử lý:

- `<sensor>`: đọc trường nhìn (fov) để dựng camera.
- `<bsdf>`: ánh xạ vật liệu Mitsuba sang hệ thống nội bộ (`diffuse`→`Lambertian`, `dielectric`→`Dielectric`, `conductor`→`Metal`), lưu theo `id`.
- `<shape>`: với khối cầu, đọc bán kính và lấy tâm từ cột tịnh tiến của ma trận biến đổi 4×4 , rồi tham chiếu vật liệu theo `id`.

Phiên bản hiện tại tập trung hỗ trợ khối cầu; các loại hình khác được bỏ qua kèm cảnh báo, và là hướng mở rộng tự nhiên trong tương lai.

3.9 Tích lũy theo thời gian và hiệu chỉnh gamma

Sau khi có màu của khung hiện tại, nhân cộng dồn vào bộ đệm tích lũy rồi chia cho số khung để giảm nhiễu (xem Chương 2). Khi camera hoặc tham số thay đổi, host đặt lại `frame_count` về 1 để loại bỏ ảnh cũ. Trước khi ghi ra PBO, màu được hiệu chỉnh gamma bằng căn bậc hai và quy về số nguyên 8-bit cho mỗi kênh RGBA.

3.10 Giao diện điều khiển tương tác

UIManager (ImGui) hiển thị bảng điều khiển cho phép tinh chỉnh các tham số camera (trường nhìn, khẩu độ, khoảng lấy nét) và theo dõi hiệu năng (FPS, thời gian mỗi khung). CameraController xử lý thao tác chuột để quay quanh (orbit), tịnh tiến (pan) và phóng to/thu nhỏ (zoom). Bất kỳ thay đổi nào từ giao diện hay camera đều kích hoạt đồng bộ camera xuống GPU và đặt lại quá trình tích lũy, bảo đảm ảnh luôn nhất quán với góc nhìn hiện tại.

Chương 4

Đánh giá

Chương này trình bày kết quả thực nghiệm của hệ thống đã hiện thực ở Chương 3. Phần đầu mô tả cấu hình phần cứng và phần mềm dùng để thử nghiệm. Phần sau phân tích các ảnh kết xuất theo từng nhóm hiệu ứng: chất lượng tổng thể, tái tạo vật liệu, độ sâu trường ảnh và nhòe chuyển động, kèm theo số liệu hiệu năng đo trực tiếp trong quá trình kết xuất.

4.1 Môi trường thử nghiệm

Toàn bộ thử nghiệm được thực hiện trên một máy tính xách tay với cấu hình nêu ở Bảng 4.1. Card đồ họa thuộc phân khúc phổ thông, dung lượng bộ nhớ video giới hạn ở 4 GB, nên kết quả hiệu năng phản ánh mức trung bình thấp của phần cứng người dùng phổ thông.

Bảng 4.1: Cấu hình môi trường thử nghiệm.

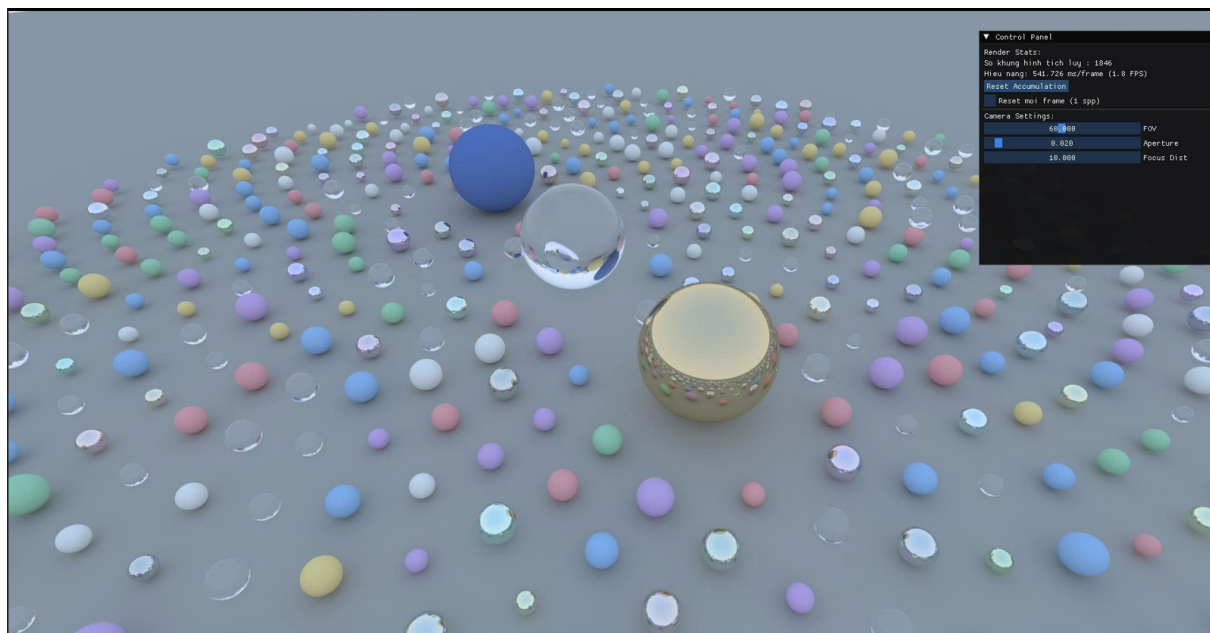
Thành phần	Thông số
CPU	AMD Ryzen 5 5600H, 6 nhân 12 luồng
GPU	NVIDIA GeForce GTX 1650, 4 GB GDDR
Bộ nhớ hệ thống	16 GB
Hệ điều hành	Windows 11, 64-bit
CUDA Toolkit	phiên bản 12.6
Trình biên dịch	MSVC kết hợp nvcc
Độ phân giải kết xuất	1920×1014 điểm ảnh

Hiệu năng được đo bằng hai chỉ số hiển thị trực tiếp trên bảng điều khiển: thời gian kết xuất trung bình của một khung hình tính bằng mili giây và số khung hình trên giây tương ứng. Mỗi khung hình lấy một mẫu cho mỗi điểm ảnh, sau đó cộng dồn vào bộ đệm tích lũy nên ảnh hội tụ dần theo thời gian. Số khung hình tích lũy được ghi nhận để thể hiện mức độ hội tụ của ảnh tại thời điểm chụp.

4.2 Chất lượng kết xuất tổng thể

Hình 4.1 là ảnh kết xuất trên cảnh có mật độ vật thể cao gồm vài trăm khối cầu với đủ ba loại vật liệu. Ảnh đạt được sau 1846 khung hình tích lũy, cho thấy thuật toán cộng dồn theo thời gian khử

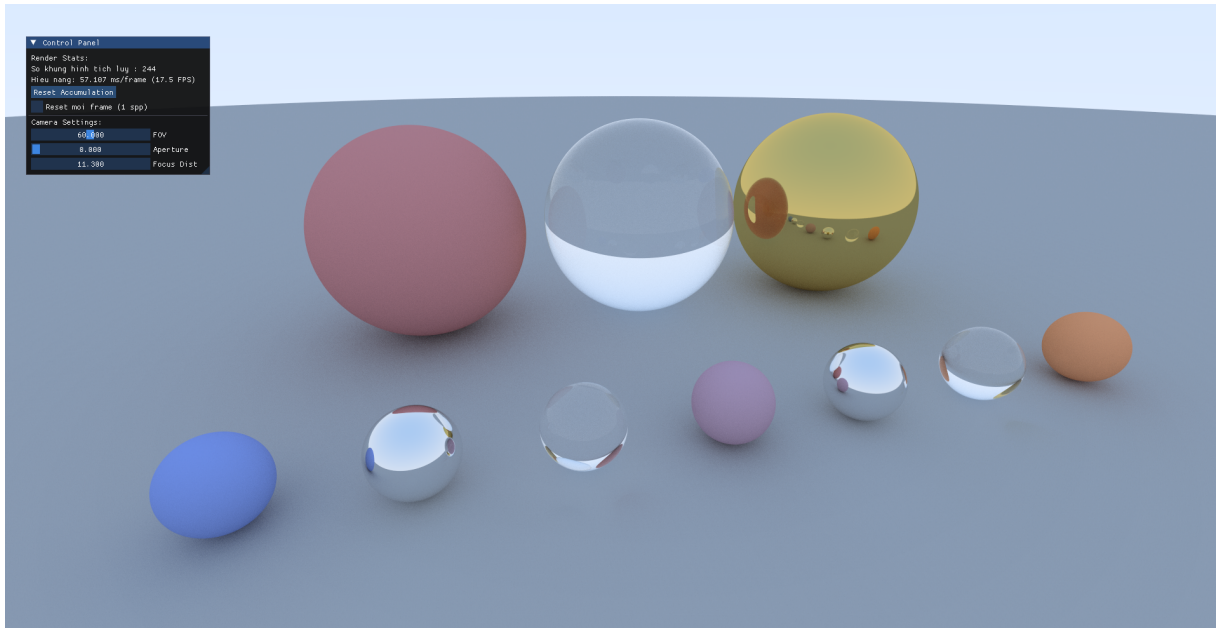
được phần lớn nhiễu Monte Carlo và tái tạo các hiệu ứng phản xạ, khúc xạ, đổ bóng mềm một cách nhất quán. Ở cảnh đông đối tượng như vậy, thời gian mỗi khung tăng lên khoảng 542 ms tương ứng 1,8 khung hình trên giây, do mỗi tia phải duyệt tuyến tính qua toàn bộ danh sách vật thể để tìm giao điểm. Đây là hệ quả trực tiếp của việc chưa dùng cấu trúc tăng tốc không gian, và là động lực cho hướng tối ưu bằng BVH nêu ở Chương 5.



Hình 4.1: Ảnh kết xuất trên cảnh mật độ cao sau 1846 khung hình tích lũy.

4.3 Tái tạo các loại vật liệu

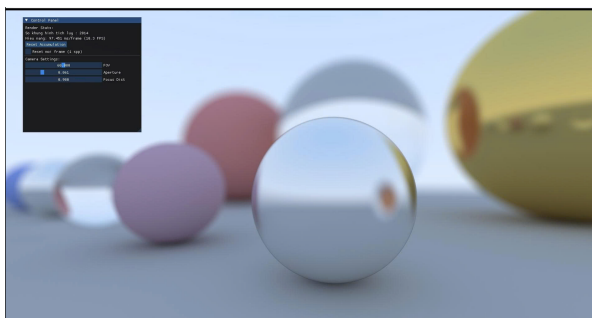
Hình 4.2 kiểm chứng riêng ba loại vật liệu đã hiện thực trên một cảnh thưa để quan sát rõ từng đặc tính. Khối cầu màu đỏ thể hiện vật liệu Lambertian với bề mặt khuếch tán đều, không phản chiếu môi trường. Khối cầu trong suốt thể hiện vật liệu điện môi với hiện tượng khúc xạ làm đảo ngược ảnh nền nhìn qua thân cầu, đồng thời có phản xạ một phần ở rìa theo xấp xỉ Schlick. Khối cầu màu vàng kim thể hiện vật liệu kim loại với phản xạ gương rõ nét, ánh lên hình ảnh các vật thể xung quanh. Tại cấu hình khẩu độ bằng 0, toàn ảnh sắc nét và hệ thống đạt khoảng 57 ms mỗi khung tương ứng 17,5 khung hình trên giây nhờ số lượng vật thể ít.



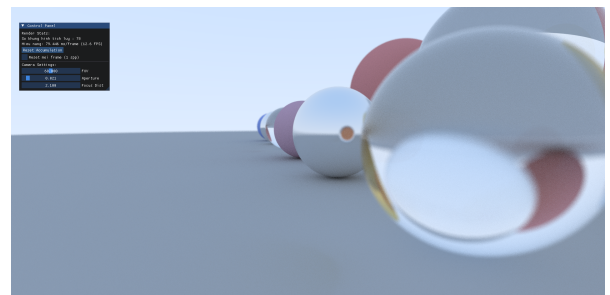
Hình 4.2: Ba loại vật liệu Lambertian, điện môi và kim loại trên cùng một cảnh.

4.4 Độ sâu trường ảnh

Mô hình camera mô phỏng thấu kính mỏng được kiểm chứng bằng cách thay đổi khẩu độ và khoảng lấy nét. Hình 4.3 minh họa hai mức khẩu độ khác nhau. Vật thể nằm đúng mặt phẳng lấy nét giữ được độ sắc nét, trong khi các vật thể ở trước và sau mặt phẳng đó bị nhòe dần theo khoảng cách. Mức độ nhòe tỉ lệ với bán kính khẩu độ: khẩu độ càng lớn thì vùng rõ nét càng hẹp và hậu cảnh càng mờ. Hiệu ứng này phát sinh tự nhiên từ việc lấy mẫu điểm xuất phát của tia trên đĩa thấu kính, không cần xử lý hậu kỳ.



(a) Khẩu độ 0,061, khoảng lấy nét 0,9.



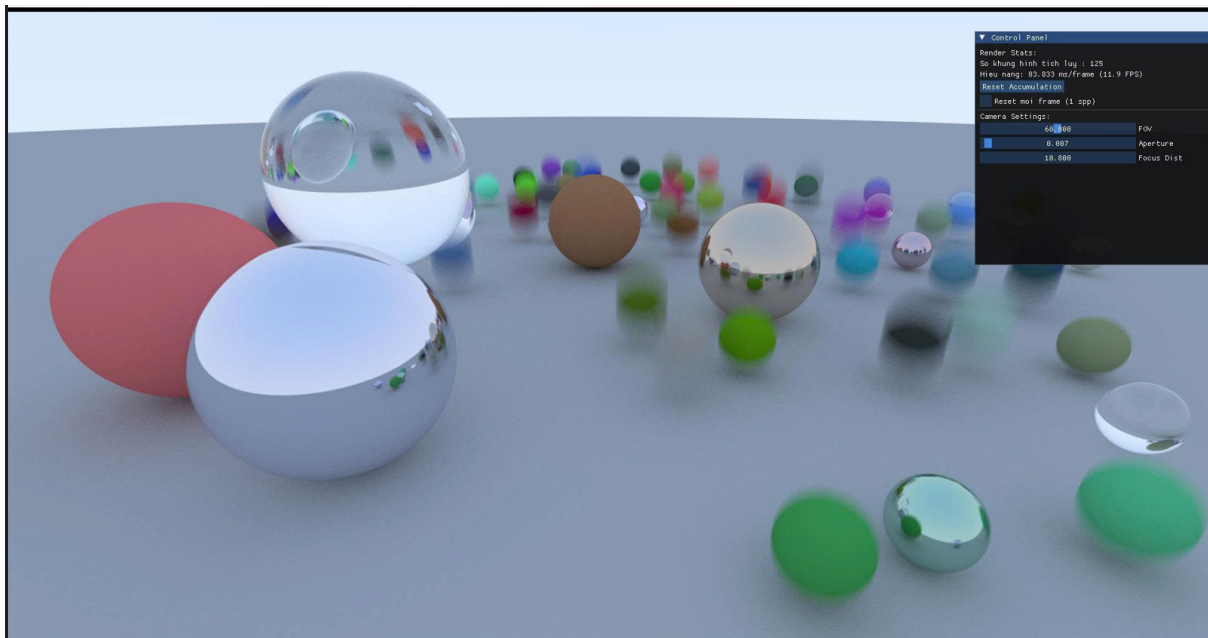
(b) Khẩu độ 0,021, khoảng lấy nét 2,1.

Hình 4.3: Hiệu ứng độ sâu trường ảnh ở hai cấu hình khẩu độ và khoảng lấy nét.

4.5 Nhòe chuyển động

Hình 4.4 kiểm chứng hiệu ứng nhòe chuyển động trên cảnh có một số khối cầu di chuyển. Các khối cầu tĩnh hiển thị sắc nét, còn các khối cầu được gán vận tốc để lại vết mờ kéo dài theo

phương chuyển động. Hiệu ứng đạt được nhờ mỗi tia mang một mốc thời gian ngẫu nhiên trong khoảng phơi sáng, và vị trí khối cầu chuyển động được nội suy theo mốc thời gian đó. Khi cộng dồn nhiều khung hình với các mốc thời gian khác nhau, vị trí của vật thể trải đều trong khoảng phơi sáng tạo thành vết nhòe liên tục.



Hình 4.4: Nhòe chuyển động trên các khối cầu được gán vận tốc.

4.6 Nhận xét chung

Qua các thử nghiệm trên, hệ thống tái tạo đầy đủ những hiệu ứng quang học mục tiêu gồm phản xạ, khúc xạ, đổ bóng mềm, độ sâu trường ảnh và nhòe chuyển động, với chất lượng hình ảnh hội tụ tốt sau vài trăm đến vài nghìn khung hình tích lũy. Về hiệu năng, hệ thống duy trì tốc độ tương tác từ 10 đến 18 khung hình trên giây với cảnh thưa, nhưng giảm rõ rệt khi mật độ vật thể tăng cao do chi phí tìm giao điểm theo phương pháp duyệt tuyến tính. Kết quả này xác nhận tính đúng đắn của hệ thống đồng thời chỉ ra nút thắt hiệu năng cần được giải quyết bằng cấu trúc tăng tốc không gian, sẽ được bàn ở Chương 5.

Chương 5

Kết luận và hướng phát triển

5.1 Kết luận

5.2 Hạn chế

5.3 Hướng phát triển trong tương lai

Một số hướng phát triển tiềm năng:

- Tích hợp lấy mẫu theo tầm quan trọng (Importance Sampling) và ước lượng sự kiện kế tiếp (Next Event Estimation) để giảm nhiễu nhanh hơn.
- Hỗ trợ thêm các mô hình BRDF/BSDF phức tạp.
- Tối ưu hóa bộ nhớ và khử nhiễu (denoising) bằng học sâu.
- Mở rộng hỗ trợ nhiều định dạng cảnh và nguồn sáng.

Tài liệu tham khảo

- [1] Wenzel Jakob, Sébastien Speierer, et al. Mitsuba 3: A retargetable forward and inverse renderer. <https://www.mitsuba-renderer.org/>, 2022.
- [2] James T. Kajiya. The rendering equation. *ACM SIGGRAPH Computer Graphics*, 20(4):143–150, 1986.
- [3] NVIDIA Corporation. Cuda c++ programming guide. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>, 2024.
- [4] Matt Pharr, Wenzel Jakob, and Greg Humphreys. *Physically Based Rendering: From Theory to Implementation*. Morgan Kaufmann, 3rd edition, 2016.
- [5] Peter Shirley, Trevor David Black, and Steve Hollasch. *Ray Tracing in One Weekend*. The ray tracing in one weekend series edition, 2020.
- [6] Peter Shirley, Trevor David Black, and Steve Hollasch. *Ray Tracing: The Rest of Your Life*. The ray tracing in one weekend series edition, 2020.
- [7] Eric Veach. *Robust Monte Carlo Methods for Light Transport Simulation*. PhD thesis, Stanford University, 1997.